

DataLab 实验报告

一、实验环境

本实验在 macOS 上完成。由于本地无法运行课程仓库中提供的 Linux ELF 可执行文件 dlc，依据课程指南中给出的替代方案，采用 GitHub Actions 进行线上评测。每次将修改推送到 GitHub 仓库后，自动化流程会在 GitHub 提供的 Linux 环境中执行 check_ops.py 和 btest，并输出每道题的规则检查与正确性测试结果。

以下实验结果的截图均来自 GitHub Actions 日志中的真实执行输出。

二、评测结果

1. check_ops.pybits.c 执行结果

```
logicalShift 0s

1 ▶ Run bash .github/scripts/autograde.sh
11 logicalShift: 6/20 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 4 4 0 logicalShift
15 Total points: 4/4
16 {"version":1,"status":"pass","max_score":4,"tests":
[{"name":"logicalShift","status":"pass","message":null,"test_code":"./check_ops.py --function logicalShift bits.c
&& ./btest -f logicalShift <stdin>","filename":"","line_no":0,"execution_time":"0.065s","score":4}]}
```

```
swapNibblePairs 0s

1 ▶ Run bash .github/scripts/autograde.sh
11 swapNibblePairs: 5/18 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 4 4 0 swapNibblePairs
15 Total points: 4/4
16 {"version":1,"status":"pass","max_score":4,"tests":
[{"name":"swapNibblePairs","status":"pass","message":null,"test_code":"./check_ops.py --function swapNibblePairs
bits.c && ./btest -f swapNibblePairs <stdin>","filename":"","line_no":0,"execution_time":"0.110s","score":4}]}
```

```
secondLowestZeroBit 0s

1 ▶ Run bash .github/scripts/autograde.sh
11 secondLowestZeroBit: 7/8 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 4 4 0 secondLowestZeroBit
15 Total points: 4/4
16 {"version":1,"status":"pass","max_score":4,"tests":
[{"name":"secondLowestZeroBit","status":"pass","message":null,"test_code":"./check_ops.py --function
secondLowestZeroBit bits.c && ./btest -f secondLowestZeroBit
<stdin>","filename":"","line_no":0,"execution_time":"0.128s","score":4}]}
```

```
oddParity 1s

1 ▶ Run bash .github/scripts/autograde.sh
11 oddParity: 12/56 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 5 5 0 oddParity
15 Total points: 5/5
16 {"version":1,"status":"pass","max_score":5,"tests":
[{"name":"oddParity","status":"pass","message":null,"test_code":"./check_ops.py --function oddParity bits.c &&
./btest -f oddParity <stdin>","filename":"","line_no":0,"execution_time":"0.200s","score":5}]}
```

✓ rotateRightBits 0s

```
1 ▶ Run bash .github/scripts/autograde.sh
11 rotateRightBits: 2/16 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 5 5 0 rotateRightBits
15 Total points: 5/5
16 {"version":1,"status":"pass","max_score":5,"tests":
  [{"name":"rotateRightBits","status":"pass","message":null,"test_code":"./check_ops.py --function rotateRightBits
  bits.c && ./btest -f rotateRightBits <stdin>","filename":"","line_no":0,"execution_time":"0.146s","score":5}]}
```

✓ roundEvenPow2 0s

```
1 ▶ Run bash .github/scripts/autograde.sh
11 roundEvenPow2: 11/24 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 5 5 0 roundEvenPow2
15 Total points: 5/5
16 {"version":1,"status":"pass","max_score":5,"tests":
  [{"name":"roundEvenPow2","status":"pass","message":null,"test_code":"./check_ops.py --function roundEvenPow2
  bits.c && ./btest -f roundEvenPow2 <stdin>","filename":"","line_no":0,"execution_time":"0.065s","score":5}]}
```

✓ midpointTowardFirst 1s

```
1 ▶ Run bash .github/scripts/autograde.sh
11 midpointTowardFirst: 21/32 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 5 5 0 midpointTowardFirst
15 Total points: 5/5
16 {"version":1,"status":"pass","max_score":5,"tests":
  [{"name":"midpointTowardFirst","status":"pass","message":null,"test_code":"./check_ops.py --function
  midpointTowardFirst bits.c && ./btest -f midpointTowardFirst
  <stdin>","filename":"","line_no":0,"execution_time":"0.214s","score":5}]}
```

✓ isBetweenEitherOrder 0s

```
1 ▶ Run bash .github/scripts/autograde.sh
11 isBetweenEitherOrder: 31/48 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 7 7 0 isBetweenEitherOrder
15 Total points: 7/7
16 {"version":1,"status":"pass","max_score":7,"tests":
  [{"name":"isBetweenEitherOrder","status":"pass","message":null,"test_code":"./check_ops.py --function
  isBetweenEitherOrder bits.c && ./btest -f isBetweenEitherOrder
  <stdin>","filename":"","line_no":0,"execution_time":"0.726s","score":7}]}
```

✓ mul5Sat 1s

```
1 ▶ Run bash .github/scripts/autograde.sh
11 mul5Sat: 27/30 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 7 7 0 mul5Sat
15 Total points: 7/7
16 {"version":1,"status":"pass","max_score":7,"tests":
  [{"name":"mul5Sat","status":"pass","message":null,"test_code":"./check_ops.py --function mul5Sat bits.c &&
  ./btest -f mul5Sat <stdin>","filename":"","line_no":0,"execution_time":"0.101s","score":7}]}
```

```

1  ▶ Run bash .github/scripts/autograde.sh
11 classifyAdd3: 38/52 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 7 7 0 classifyAdd3
15 Total points: 7/7
16 {"version":1,"status":"pass","max_score":7,"tests":
  [{"name":"classifyAdd3","status":"pass","message":null,"test_code":"./check_ops.py --function classifyAdd3 bits.c
  && ./btest -f classifyAdd3 <stdin>","filename":"","line_no":0,"execution_time":"0.797s","score":7}]}

```

```

1  ▶ Run bash .github/scripts/autograde.sh
11 floatScaleThreeHalves: 46/60 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 7 7 0 floatScaleThreeHalves
15 Total points: 7/7
16 {"version":1,"status":"pass","max_score":7,"tests":
  [{"name":"floatScaleThreeHalves","status":"pass","message":null,"test_code":"./check_ops.py --function
  floatScaleThreeHalves bits.c && ./btest -f floatScaleThreeHalves
  <stdin>","filename":"","line_no":0,"execution_time":"0.140s","score":7}]}

```

```

1  ▶ Run bash .github/scripts/autograde.sh
11 floatRoundEven: 27/65 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 10 10 0 floatRoundEven
15 Total points: 10/10
16 {"version":1,"status":"pass","max_score":10,"tests":
  [{"name":"floatRoundEven","status":"pass","message":null,"test_code":"./check_ops.py --function floatRoundEven
  bits.c && ./btest -f floatRoundEven <stdin>","filename":"","line_no":0,"execution_time":"0.161s","score":10}]}

```

```

1  ▶ Run bash .github/scripts/autograde.sh
11 float_i2f: 26/40 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 10 10 0 float_i2f
15 Total points: 10/10
16 {"version":1,"status":"pass","max_score":10,"tests":
  [{"name":"float_i2f","status":"pass","message":null,"test_code":"./check_ops.py --function float_i2f bits.c &&
  ./btest -f float_i2f <stdin>","filename":"","line_no":0,"execution_time":"0.149s","score":10}]}

```

```

1  ▶ Run bash .github/scripts/autograde.sh
11 bitCount: 8/40 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 10 10 0 bitCount
15 Total points: 10/10
16 {"version":1,"status":"pass","max_score":10,"tests":
  [{"name":"bitCount","status":"pass","message":null,"test_code":"./check_ops.py --function bitCount bits.c &&
  ./btest -f bitCount <stdin>","filename":"","line_no":0,"execution_time":"0.173s","score":10}]}

```

```
bitReverse 1s
1 ▶ Run bash .github/scripts/autograde.sh
11 bitReverse: 34/34 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 10 10 0 bitReverse
15 Total points: 10/10
16 {"version":1,"status":"pass","max_score":10,"tests":
[{"name":"bitReverse","status":"pass","message":null,"test_code":"./check_ops.py --function bitReverse bits.c && ./btest -f bitReverse <stdin>","filename":"","line_no":0,"execution_time":"0.172s","score":10}]}
```

```
bitXor 0s
1 ▶ Run bash .github/scripts/autograde.sh
11 bitXor: 8/8 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 2 2 0 bitXor
15 Total points: 2/2
16 {"version":1,"status":"pass","max_score":2,"tests":
[{"name":"bitXor","status":"pass","message":null,"test_code":"./check_ops.py --function bitXor bits.c && ./btest -f bitXor <stdin>","filename":"","line_no":0,"execution_time":"0.163s","score":2}]}
```

```
copyByteWithin 0s
1 ▶ Run bash .github/scripts/autograde.sh
11 copyByteWithin: 10/12 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 4 4 0 copyByteWithin
15 Total points: 4/4
16 {"version":1,"status":"pass","max_score":4,"tests":
[{"name":"copyByteWithin","status":"pass","message":null,"test_code":"./check_ops.py --function copyByteWithin bits.c && ./btest -f copyByteWithin <stdin>","filename":"","line_no":0,"execution_time":"0.065s","score":4}]}
```

```
negativePart 1s
1 ▶ Run bash .github/scripts/autograde.sh
11 negativePart: 4/6 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 3 3 0 negativePart
15 Total points: 3/3
16 {"version":1,"status":"pass","max_score":3,"tests":
[{"name":"negativePart","status":"pass","message":null,"test_code":"./check_ops.py --function negativePart bits.c && ./btest -f negativePart <stdin>","filename":"","line_no":0,"execution_time":"0.095s","score":3}]}
```

```
signMask 0s
1 ▶ Run bash .github/scripts/autograde.sh
11 signMask: 1/2 operators
12 All requested functions passed operator checks.
13 Score Rating Errors Function
14 1 1 0 signMask
15 Total points: 1/1
16 {"version":1,"status":"pass","max_score":1,"tests":
[{"name":"signMask","status":"pass","message":null,"test_code":"./check_ops.py --function signMask bits.c && ./btest -f signMask <stdin>","filename":"","line_no":0,"execution_time":"0.064s","score":1}]}
```

2. btest 执行结果

```

238  🏆 Grand total tests passed: 19/19
239
240  Notice: Points 110/110
241  Notice: {"totalPoints":110,"maxPoints":110}

```

三、各函数实现思路

P1 signMask

返回 $1 \ll 31$ ，即最高位为 1、其余为 0 的掩码 0x80000000。

P2 bitXor

用 $(\sim x \& y) \mid (x \& \sim y)$ 表示异或，再用德摩根律 $a \mid b = \sim(\sim a \& \sim b)$ 把 $\mid$ 消去，得到 $\sim(\sim(\sim x \& y) \& \sim(x \& \sim y))$ ，只使用 $\sim$ 和 $\&$ 。

P3 negativePart

$\text{mask} = x \gg 31$ 得到符号掩码：x 为负时全 1，非负时全 0。用 $\text{negx} = \sim x + 1$ 计算 -x，返回 $\text{negx} \& \text{mask}$ 。x 为负时返回 -x，非负时被掩码清零。

P4 copyByteWithin

$\text{clearMask} = \sim(0xFF \ll (\text{dst} \ll 3))$ 清除目标字节的位置。 $\text{srcByte} = (x \gg (\text{src} \ll 3)) \& 0xFF$ 取出源字节。最后返回 $(x \& \text{clearMask}) \mid (\text{srcByte} \ll (\text{dst} \ll 3))$ ，把源字节放入目标位置并保留其余字节。

P5 logicalShift

先做算术右移 $x \gg n$ ，此时高位会补入符号位。用掩码 $\sim((1 \ll 31) \gg n \ll 1)$ 得到"高 n 位为 0、其余为 1"的掩码，与结果做 $\&$ 将补入的符号位清零，实现逻辑右移。

P6 swapNibblePairs

用 $0x0F \mid (0x0F \ll 8)$ 再 $\mid (m \ll 16)$ 构造掩码 0x0F0F0F0F。 $(x \& m) \ll 4$ 把每字节低 4 位搬到高 4 位； $((x \& \sim m) \gg 4) \& m$ 把每字节高 4 位搬到低 4 位，并再用掩码清掉算术右移补入的符号位。两者做 $\mid$ 合并。

P7 secondLowestZeroBit

$\sim x \& (x + 1)$ 定位最低的 0 位。 $x \mid \text{lowest}$ 把它置 1，则原来的第 2 个 0 位成为新值的最低 0 位。再对新值做一次 $\sim x2 \& (x2 + 1)$ 即得结果；不存在第 2 个 0 时结果自然为 0。

P8 oddParity

用折半异或把 32 位的奇偶性合并到最低位：依次 $x \wedge= x \gg 16, \gg 8, \gg 4, \gg 2, \gg 1$ 。异或的性质保证最低位等于 32 位中 1 的个数的奇偶性。题目要求偶数个 1 返回 1、奇数个 1 返回 0，最后用 $(x \& 1) \wedge 1$ 取反。

P9 rotateRightBits

先 $n = n \& 31$ 处理 $n \geq 32$ 的情况。 $s = 33 + \sim n$ 即 $32 - n$ 。 $x \ll s$ 把低 n 位搬到最高位； $(x \gg n) \& \sim(\sim 0 \ll s)$ 把高 $(32 - n)$ 位搬到低位，并清掉算术右移补入的符号位。两者做 $|$ 。

P10 roundEvenPow2

用偏移量法实现"四舍六入五成双"。 $half = 1 \ll (n-1)$ 是半个单位， $lowBit = (x \gg n) \& 1$ 是商的奇偶。偏移量取 $half - 1 + lowBit$ ，加到 x 后再 $\gg n \ll n$ 清掉低 n 位。中点时偏移量让结果刚好落在偶数商对应的倍数上。

P11 midpointTowardFirst

$avg = (x \& y) + ((x \wedge y) \gg 1)$ 在不溢出的前提下算出向下取整的平均值。 $isOdd = (x \wedge y) \& 1$ 判断是否为半整数。 $x > y$ 的判断带符号处理： $d = sx \wedge sy$ 表示两数是否异号， $sub = (x + (\sim y + 1)) \gg 31$ 是 $x - y$ 的符号位， $xGtY = (d \& \sim sx) | ((d \wedge 1) \& \sim sub)$ 综合两种情况。最后 $avg + (isOdd \& xGtY)$ ，中点且 $x > y$ 时加 1，使中点偏向第一个参数。

P12 isBetweenEitherOrder

分别计算 $x - a$ 与 $x - b$ 的真实符号。当 x 与端点同号 ($da = 0$ 或 $db = 0$) 时，差值不会溢出，直接取差值符号位；异号时差值的真实符号就等于 x 的符号。 ra 、 rb 即两个差值的真实符号。若 $ra \wedge rb = 1$ (两差值异号)，或至少一个差值为 0，说明 x 落在区间内。

P13 mul5Sat

先算 $x4 = x \ll 2$ 与 $x5 = x4 + x$ 。 $x \ll 2$ 的溢出有两种情况：符号位变化 ($ovf4a$)，或 x 的高两位不是全 0 / 全 1 ($hi = x \gg 30$ ，用 $!!hi \& !!\sim hi$ 判断)。 $x4 + x$ 的溢出用符号位变化判断 ($ovf5$)。任一溢出时，用 $sat = sx \wedge \sim(1 \ll 31)$ 得到： x 非负时 INT_MAX ， x 为负时 INT_MIN 。最后用掩码 $m = \sim ovf + 1$ 在溢出时选 sat 、否则选 $x5$ 。

P14 classifyAdd3

分两步相加。先算 $s1 = x + y$ ，用 $ovf1 = (sx \wedge ss1) \& (sy \wedge ss1)$ 判断溢出；若 x 、 y 都非负且溢出则为正溢出 $pos1$ ，都非负同理为 $neg1$ 。再把 $s1 + z$ 用同样方式得到 $ovf2$ 、 $pos2$ 、 $neg2$ 。 $pos = pos1 | pos2$ 、 $neg = neg1 | neg2$ ；若两者同时成立视为互相抵消。最终 $res + (\sim res2 + 1)$ 返回 1、-1 或 0。

P15 floatScaleThreeHalves

分离符号、阶码、尾数。NaN / Inf 直接返回；0 保持符号返回。非规格化数： $m = frac * 3$ ，右移 1 位，若最低位为 1 且商的最低位也是 1 则按就近取偶进位；若进位到 24 位则提升为最小规格化数。规格化数： $m3 = mant + (mant \ll 1)$ 代替乘 3 ($mant$ 为 24 位)。若 $m3$ 是 26 位 ($m3 \& 0x20000000$)，右移 2 位，用余数 $r = m3 \& 3$ 判断 $r > 2 \parallel (r == 2 \& \& q \text{ 为奇})$ 时进位，指数加 1；否则右移 1 位，用最低位判断是否进位，若进位到 25 位再右移 1 位、指数加 1。最后检查阶码是否溢出。

P16 floatRoundEven

分离符号、阶码、尾数。NaN / Inf 原样返回。 $e < -1$ 返回带符号 0； $e == -1$ 时若尾数为 0 (即恰好 ± 0.5) 按就近取偶返回 ± 0 ，否则 $(0.5 < |x| < 1)$ 返回 ± 1 。 $e \geq 23$ 时已

是整数，直接返回。其余情况： $ip = mant \gg shift$ 是整数部分， $fp = mant \& ((1 \ll shift) - 1)$ 是小数部分， $half = 1 \ll (shift - 1)$ 是中点。按 $fp > half \parallel (fp == half \&\& ip \text{ 为奇})$ 决定进位。然后用 $ip \ll (23 - e)$ 重新归一化；若进位到 25 位则右移 1 位、指数加 1，再拼成结果。

P17 float_i2f

0 和 INT_MIN 特判。取绝对值后从高到低找到最高有效位得到指数 e 。 $e \leq 23$ 时把尾数整体左移到高位； $e > 23$ 时右移丢弃低位，并用余数与 $half$ 比较做就近取偶舍入，若进位到 24 位则指数加 1。最后拼成单精度浮点表示。

P18 bitCount

折半统计。用 0x55555555、0x33333333、0x0F0F0F0F、0x00FF00FF、0x0000FFFF 这几组掩码，依次把相邻 1、2、4、8、16 位的 1 的个数两两相加，五步后即得 32 位中 1 的总数。每个掩码由 0x55、0x33、0x0F、0xFF 通过移位和 | 拼出。

P19 bitReverse

采用分治交换。先用 $m16 = 0xFF \mid (0xFF \ll 8)$ 得到 0x0000FFFF，再用 $m8 = m16 \wedge (m16 \ll 8)$ 、 $m4 = m8 \wedge (m8 \ll 4)$ 、 $m2 = m4 \wedge (m4 \ll 2)$ 、 $m1 = m2 \wedge (m2 \ll 1)$ 逐层异或推出各层掩码。然后依次交换相邻 1、2、4、8、16 位，每步用掩码保留一半位再移位合并，完成 32 位反转。

四、实验总结

本次实验共 19 道题，覆盖了整数补码表示、位运算、掩码构造、溢出判断、移位规则，以及 IEEE754 单精度浮点数的舍入与特殊值处理。

整数题的核心思路是用位运算构造合适的掩码来“选值”，避免使用分支和比较。常见的坑包括算术右移补符号位导致的污染、 $n=0$ 时移位 32 位的未定义行为、溢出判断中符号位失效等。浮点题则需要先拆出符号、阶码和尾数，再针对规格化数、非规格化数、NaN 与 Inf 分别处理，舍入部分要严格按照“就近取偶”的规则实现。

通过在 GitHubActions 上反复提交、对照失败用例定位问题，逐步修正了各题实现，最终取得 110/110。